

Tài liệu Module FaceDetector

Lê Hoàng An

May 31, 2025

1 Giới thiệu

Tài liệu này mô tả các luồng xử lý của class FaceDetector, một lớp được thiết kế để phát hiện khuôn mặt trong ảnh, tối ưu cho dataset WIDER FACE. Class hỗ trợ hai phương pháp phát hiện khuôn mặt: MTCNN (dựa trên thư viện facenet-pytorch) và OpenCV Haar Cascade. Tài liệu bao gồm các luồng chính, vòng đời (lifecycle), luồng xác thực đầu vào, và các luồng chi tiết liên quan đến xử lý phát hiện khuôn mặt.

2 Flow Chính của Module

Module FaceDetector hoạt động theo các bước chính sau:

1. **Khởi tạo:** Khởi tạo đối tượng FaceDetector với các tham số như phương pháp phát hiện (method), thiết bị (device), và các ngưỡng xử lý.
2. **Phát hiện khuôn mặt:** Sử dụng phương pháp được chọn (MTCNN hoặc OpenCV) để phát hiện khuôn mặt trong frame ảnh đầu vào.
3. **Xử lý sau phát hiện:** Áp dụng ngưỡng tin cậy (confidence_threshold) và Non-Maximum Suppression (NMS) để lọc các hộp giới hạn (bounding boxes).
4. **Đánh giá hiệu suất:** Đánh giá trên dataset với các chỉ số như số lượng khuôn mặt phát hiện, thời gian xử lý trung bình, và tỷ lệ phát hiện.

3 Lifecycle của Module

Vòng đời của một đối tượng FaceDetector bao gồm các giai đoạn sau:

1. Khởi tạo (`__init__`):

- Nhận các tham số cấu hình: `method` (MTCNN hoặc OpenCV), `device` (CPU/GPU), `confidence_threshold`, `nms_threshold`, và `min_face_size`.
- Nếu `method='mtcnn'`, tải mô hình MTCNN từ `facenet-pytorch`. Nếu thư viện chưa được cài đặt, tự động cài đặt.
- Nếu `method='opencv'`, tải Haar Cascade từ OpenCV.
- Nếu phương pháp không được hỗ trợ,□ ra lỗi `ValueError`.

2. Sử dụng (`detect_faces`):

- Nhận frame ảnh và tùy chọn trả về landmarks.
- Chuyển đổi frame sang định dạng phù hợp (RGB cho MTCNN, grayscale cho OpenCV).
- Thực hiện phát hiện khuôn mặt và lọc kết quả bằng ngưỡng tin cậy và NMS.

3. Đánh giá (`evaluate_on_dataset`):

- Xử lý một tập hợp ảnh từ `dataset_loader`.
- Tính toán các chỉ số hiệu suất như số lượng khuôn mặt phát hiện, thời gian xử lý, và tỷ lệ phát hiện.

4. **Kết thúc:** Không có phương thức hủy rõ ràng, đối tượng được giải phóng tự động bởi Python garbage collector khi không còn được tham chiếu.

4 Input Validation Flow

Luồng xác thực đầu vào được thực hiện như sau:

1. Khởi tạo (`__init__`):

- Kiểm tra `method`: Nếu không phải `'mtcnn'` hoặc `'opencv'`, ném `ValueError`.
- Kiểm tra thư viện `facenet-pytorch` cho MTCNN: Nếu thiếu, tự động cài đặt.
- Kiểm tra đường dẫn cascade file cho OpenCV: Sử dụng đường dẫn mặc định từ `cv2.data.harcascades`.

2. Phát hiện khuôn mặt (**detect_faces**):

- Kiểm tra frame đầu vào: Frame phải là một mảng NumPy hợp lệ (được xử lý bởi OpenCV hoặc MTCNN).
- Kiểm tra định dạng màu: Chuyển đổi tự động từ BGR sang RGB (cho MTCNN) hoặc grayscale (cho OpenCV).
- Kiểm tra kết quả phát hiện: Nếu không phát hiện được khuôn mặt, trả về mảng rỗng.

3. Đánh giá (**evaluate_on_dataset**):

- Kiểm tra dataset_loader: Đảm bảo image_paths được tải trước khi xử lý.
- Kiểm tra ảnh đầu vào: Nếu `cv2.imread` trả về None, bỏ qua ảnh đó.
- Xử lý lỗi: Bắt các ngoại lệ trong quá trình xử lý ảnh và ghi log lỗi.

5 Các Flow Chi Tiết

Dưới đây là các luồng chi tiết liên quan đến các phương thức chính:

5.1 Phát hiện khuôn mặt (**detect_faces**)

- **Đầu vào:** Frame ảnh (NumPy array), `return_landmarks` (boolean).
- **Xử lý:**
 - Nếu `method='mtcnn'`:
 - * Chuyển frame sang RGB.
 - * Gọi `self.detector.detect` để lấy hộp giới hạn, xác suất, và landmarks (nếu yêu cầu).
 - * Lọc kết quả bằng `confidence_threshold`.
 - * Áp dụng NMS thông qua `_apply_nms`.
 - Nếu `method='opencv'`:
 - * Chuyển frame sang grayscale.
 - * Gọi `detectMultiScale` để phát hiện khuôn mặt.

- * Chuyển đổi kết quả sang định dạng (x1, y1, x2, y2).
- **Đầu ra:** Hộp giới hạn (boxes), xác suất (probs), và landmarks (nếu có).

5.2 Non-Maximum Suppression (`_apply_nms`)

- **Đầu vào:** Hộp giới hạn, xác suất, và `iou_threshold`.
- **Xử lý:**
 - Sắp xếp hộp theo xác suất giảm dần.
 - Lặp lại:
 - * Chọn hộp có xác suất cao nhất.
 - * Tính IoU với các hộp còn lại.
 - * Loại bỏ các hộp có IoU vượt `iou_threshold`.
- **Đầu ra:** Danh sách hộp và xác suất sau khi lọc NMS.

5.3 Tính IoU (`_calculate_iou`)

- **Đầu vào:** Một hộp và danh sách các hộp.
- **Xử lý:**
 - Tính tọa độ giao nhau (intersection) giữa các hộp.
 - Tính diện tích giao nhau và hợp (union).
 - Tính $\text{IoU} = \text{intersection} / \text{union}$.
- **Đầu ra:** Mảng các giá trị IoU.

5.4 Đánh giá trên dataset (`evaluate_on_dataset`)

- **Đầu vào:** `dataset_loader`, `sample_size`.
- **Xử lý:**
 - Tải danh sách ảnh từ `dataset_loader`.
 - Lặp qua từng ảnh:
 - * Đọc ảnh bằng `cv2.imread`.

- * Gọi `detect_faces` để phát hiện khuôn mặt.
- * Ghi lại thời gian xử lý và số lượng khuôn mặt.
- Tính toán các chỉ số: số ảnh xử lý, số khuôn mặt phát hiện, thời gian trung bình, tỷ lệ phát hiện.
- **Đầu ra:** Dictionary chứa các chỉ số hiệu suất.

6 Kết luận

Class `FaceDetector` cung cấp một giải pháp linh hoạt để phát hiện khuôn mặt với hai phương pháp MTCNN và OpenCV. Các luồng xử lý được thiết kế để đảm bảo tính mạnh mẽ, bao gồm xác thực đầu vào, lọc kết quả, và đánh giá hiệu suất. Để sử dụng hiệu quả, cần đảm bảo các thư viện (`facenet-pytorch`, `opencv-python`) được cài đặt và dataset đầu vào được chuẩn bị đúng cách.